

Seguimiento V – Electrónica Digital II

Monitoreo de temperatura, humedad y acelerómetro (ACC)

Daniela Diaz Ossa
Laura Ximena Uribe
Jailler Dany Gaviria

20 de mayo de 2026

Índice

1. Descripción del Proyecto	2
2. Aplicaciones del Sistema	2
3. Funcionalidades Principales	2
4. Objetivos del Sistema	2
5. Componentes del Sistema	2
6. Detección de Movimiento	3
7. Funcionamiento del Sistema	3
7.1. Inicialización	3
7.2. Monitoreo Continuo	3
8. Sistema de Alertas	4
8.1. Alertas Ambientales	4
8.2. Alertas de Movimiento	4
9. Servidor Web	4
10.Bot de Telegram	4
11.Código del Proyecto	5
12.Conclusión	16

1. Descripción del Proyecto

El proyecto consiste en el desarrollo de un nodo IoT basado en ESP32 orientado al monitoreo ambiental y contextual en entornos biomédicos. El sistema permite medir variables como la temperatura y la humedad relativa, así como el movimiento o el estado estático del dispositivo.

2. Aplicaciones del Sistema

- Monitoreo de condiciones ambientales en habitaciones hospitalarias.
- Supervisión de incubadoras o áreas de aislamiento.
- Detección de movimiento en dispositivos médicos portátiles o pacientes.

El monitoreo de factores ambientales en entornos hospitalarios y áreas de manejo especiales sirve para garantizar unas condiciones ideales para equipos, medicamentos, reactivos y pacientes.

3. Funcionalidades Principales

- Visualización en tiempo real mediante servidor web embebido.
- Comunicación remota mediante bot de Telegram.
- Sistema de alertas multimodal (visual, sonora y remota).

4. Objetivos del Sistema

1. Configurar un ESP32 con MicroPython y conectarlo a una red Wi-Fi.
2. Implementar un servidor web embebido con actualización automática de variables.
3. Medir temperatura y humedad usando DHT11/DHT22.
4. Integrar un sensor MPU6050 para detectar movimiento.
5. Implementar un sistema de alertas configurable.
6. Integrar un bot de Telegram para monitoreo remoto y control del sistema.

5. Componentes del Sistema

- ESP32
- DHT11 o DHT22
- MPU6050 (acelerómetro + giroscopio)
- Pulsador (botón de pánico)

- Buzzer pasivo
- Protoboard y jumpers
- Celular con Telegram
- Red Wi-Fi
- PC

6. Detección de Movimiento

El sensor MPU6050 permitirá identificar estados de reposo, movimiento moderado y movimiento brusco mediante el análisis de la magnitud del vector de aceleración:

$$|\mathbf{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2} \quad (1)$$

Comparación contra umbral:

- Si $|\mathbf{a}| \approx g \Rightarrow$ reposo.
- Si $|\mathbf{a}| > g + \delta \Rightarrow$ movimiento.

7. Funcionamiento del Sistema

7.1. Inicialización

- El ESP32 se conecta automáticamente a la red Wi-Fi.
- El ESP32 debe arrancar automáticamente al conectarse a una fuente de alimentación.
- Se inicia el servidor web embebido.
- El bot de Telegram envía la IP del dispositivo.
- Se inicializa la lectura de sensores DHT y MPU6050.

7.2. Monitoreo Continuo

- Temperatura
- Humedad
- Estado de movimiento (reposo / movimiento)

8. Sistema de Alertas

8.1. Alertas Ambientales

- Alerta térmica por temperatura baja o alta.
- Alerta por humedad baja o alta.
- Alerta combinada.

8.2. Alertas de Movimiento

- Movimiento detectado.
- Movimiento brusco.

Las alertas deben implementarse de forma multimodal mediante servidor web, buzzer y notificaciones remotas en Telegram.

9. Servidor Web

Variables mostradas en la interfaz web:

- Temperatura
- Humedad
- Estado de movimiento
- Estado de alarmas
- Umbrales configurados

10. Bot de Telegram

Funciones disponibles:

- Consultar temperatura, humedad y movimiento.
- Consultar umbrales configurados.
- Recibir alarmas de temperatura, humedad y movimiento.

11. Código del Proyecto

```

1  # =====
2  # PROYECTO: Nodo IoT Biomedico
3  # ESP32 + DHT11 + MPU6050 + Telegram + Web
4  # THONNY + ESP32 REAL
5  # VERSION FINAL ESTABLE
6  # =====
7
8  import network
9  import time
10 import machine
11 import dht
12 import struct
13 import math
14 import urequests
15 import socket
16
17 print("\n#####")
18 print(" IOT BIOMEDICO - DIGITAL II")
19 print(" Iniciando sistema...")
20 print("#####\n")
21
22 # =====
23 # UMBRALES
24 # =====
25
26 T_MIN = 18
27 T_MAX = 30
28
29 H_MIN = 30
30 H_MAX = 70
31
32 A_MOV = 1.15
33 A_BRUS = 1.80
34
35 # =====
36 # TELEGRAM
37 # =====
38
39 TOKEN = "8768289337:AAFz8XQ2I69Zfo_L1PAU0iStHFjzjq4Po3Q"
40 CHAT_ID = "8657310658"
41
42 ultimo_mensaje = ""
43 ultimo_envio = 0
44 ultimo_update_id = 0
45
46 # =====
47 # VARIABLES
48 # =====
49

```

```
50 temp = 25
51 hum = 50
52 mag = 1.0
53
54 emov = "Reposo"
55
56 al_t = ""
57 al_h = ""
58 al_m = ""
59
60 al_p = False
61
62 panico_activo = False
63 _panico_flag = False
64
65 # =====
66 # DHT11
67 # =====
68
69 DHT_PIN = 18
70
71 _dht = dht.DHT11(machine.Pin(DHT_PIN))
72
73 print("[HW] DHT11 GPIO", DHT_PIN)
74
75 # =====
76 # MPU6050
77 # =====
78
79 try:
80     _i2c = machine.I2C(
81         0,
82         scl=machine.Pin(22),
83         sda=machine.Pin(21),
84         freq=100000
85     )
86     print("[HW] I2C Hardware")
87 except:
88     _i2c = machine.SoftI2C(
89         scl=machine.Pin(22),
90         sda=machine.Pin(21),
91         freq=100000
92     )
93     print("[HW] I2C Software")
94
95 # =====
96 # BUZZER
97 # =====
98
99 _bp = machine.Pin(5, machine.Pin.OUT)
100 _bz = machine.PWM(_bp)
```

```

101 _bz.freq(1000)
102
103 try:
104     _bz.duty_u16(0)
105 except:
106     _bz.duty(0)
107
108 print("[HW] Buzzer GPIO5")
109
110 # =====
111 # BOTON PANICO
112 # =====
113
114 ultimo_panico = 0
115
116 def _isr_panico(pin):
117     global _panico_flag
118     global ultimo_panico
119     ahora = time.time()
120     if time.time_diff(ahora, ultimo_panico) > 180:
121         _panico_flag = True
122         ultimo_panico = ahora
123
124 _btn = machine.Pin(15, machine.Pin.IN, machine.Pin.PULL_UP)
125 _btn.irq(trigger=machine.Pin.IRQ_FALLING, handler=_isr_panico)
126
127 print("[HW] Boton GPIO15")
128
129 # =====
130 # TELEGRAM ENVIAR
131 # =====
132
133 def telegram(msg):
134     global ultimo_mensaje
135     global ultimo_envio
136     try:
137         ahora = time.time()
138         if ahora - ultimo_envio < 3:
139             return
140         if msg == ultimo_mensaje:
141             return
142         ultimo_mensaje = msg
143         ultimo_envio = ahora
144         msg = msg.replace(" ", "%20")
145         msg = msg.replace("\n", "%0A")
146         url = (
147             "https://api.telegram.org/bot"
148             + TOKEN
149             + "/sendMessage?chat_id="
150             + CHAT_ID
151             + "&text="

```



```

152         + msg
153     )
154     r = urequests.get(url)
155     print("[TELEGRAM OK]")
156     r.close()
157 except Exception as e:
158     print("[TELEGRAM ERROR]", e)
159
160 # =====
161 # TELEGRAM RECIBIR
162 # =====
163
164 def revisar_telegram():
165     global ultimo_update_id
166     try:
167         url = (
168             "https://api.telegram.org/bot"
169             + TOKEN
170             + "/getUpdates?offset="
171             + str(ultimo_update_id + 1)
172         )
173         r = urequests.get(url)
174         datos = r.json()
175         r.close()
176         if datos["ok"]:
177             for mensaje in datos["result"]:
178                 ultimo_update_id = mensaje["update_id"]
179                 texto = ""
180                 try:
181                     texto = mensaje["message"]["text"]
182                 except:
183                     pass
184                 texto = texto.lower()
185                 print("[TELEGRAM RX]", texto)
186                 if texto == "/estado":
187                     respuesta = (
188                         "ESTADO ACTUAL\n\n"
189                         + "Temperatura: " + str(temp) + " C\n"
190                         + "Humedad: " + str(hum) + " %\n"
191                         + "Movimiento: " + emov + "\n"
192                         + "Magnitud: " + str(round(mag, 2)) + " g\n"
193                         + "Panico: " + ("ACTIVO" if
194                                     panico_activo else "NO")
195                     )
196                     telegram(respuesta)
197                 elif texto == "/temp":
198                     telegram("Temperatura actual: " + str(temp) + " C")
199                 elif texto == "/hum":
200                     telegram("Humedad actual: " + str(hum) + " %")

```

```

200         elif texto == "/mov":
201             telegram(
202                 "Movimiento actual: " + emov
203                 + "\nMagnitud: " + str(round(mag, 2)) + " g
204             )
205         elif texto == "/panic":
206             telegram("Estado panico: " + ("ACTIVO" if
207                 panico_activo else "NO"))
208         elif texto == "/help":
209             telegram(
210                 "COMANDOS DISPONIBLES\n\n"
211                 + "/estado\n/temp\n/hum\n/mov\n/panic"
212             )
213     except Exception as e:
214         print("[TELEGRAM BOT ERROR]", e)
215
216 # =====
217 # MPU6050
218 # =====
219
220 def init_mpu():
221     try:
222         devs = _i2c.scan()
223         print("[MPU] I2C:", [hex(d) for d in devs])
224         if 0x68 in devs:
225             _i2c.writeto_mem(0x68, 0x6B, b'\x00')
226             time.sleep_ms(200)
227             print("[MPU] MPU6050 OK")
228             return True
229         print("[MPU] No detectado")
230         return False
231     except Exception as e:
232         print("[MPU ERROR]", e)
233         return False
234
235 def leer_acc():
236     try:
237         d = _i2c.readfrom_mem(0x68, 0x3B, 6)
238         ax, ay, az = struct.unpack(">hhh", d)
239         ax /= 16384.0
240         ay /= 16384.0
241         az /= 16384.0
242         magnitud = math.sqrt(ax*ax + ay*ay + az*az)
243         return magnitud
244     except Exception as e:
245         print("[MPU ERROR]", e)
246         return 1.0
247
248 def clasificar(m):
249     print("[MPU MAG]", round(m, 3))

```

```

249     if m >= A_BRUS:
250         return "Movimiento Brusco"
251     if m >= A_MOV:
252         return "Movimiento"
253     return "Reposo"
254
255     # =====
256     # DHT11
257     # =====
258
259 def leer_dht():
260     try:
261         _dht.measure()
262         t = _dht.temperature()
263         h = _dht.humidity()
264         print("[DHT RAW]", t, h)
265         if t is None or h is None:
266             print("[DHT] Lectura nula")
267             return None, None
268         if t < -10 or t > 60:
269             print("[DHT] Temperatura invalida")
270             return None, None
271         if h < 0 or h > 100:
272             print("[DHT] Humedad invalida")
273             return None, None
274         return t, h
275     except Exception as e:
276         print("[DHT ERROR]", e)
277         return None, None
278
279     # =====
280     # BUZZER
281     # =====
282
283 def buzzer_on():
284     try:
285         _bz.duty_u16(9000)
286     except:
287         _bz.duty(140)
288
289 def buzzer_off():
290     try:
291         _bz.duty_u16(0)
292     except:
293         _bz.duty(0)
294
295     # =====
296     # ALARMAS
297     # =====
298
299 def evaluar():

```

```
300 global al_t, al_h, al_m, al_p
301 global _panico_flag, panico_activo
302
303 lista_alertas = []
304
305 if _panico_flag:
306     _panico_flag = False
307     panico_activo = not panico_activo
308     if panico_activo:
309         print("\n*** PANICO ACTIVADO ***\n")
310         telegram("PANICO ACTIVADO")
311     else:
312         print("\n*** PANICO DESACTIVADO ***\n")
313         telegram("PANICO DESACTIVADO")
314
315 if temp > T_MAX:
316     al_t = "TEMP ALTA"
317     lista_alertas.append("Temperatura alta: " + str(temp) + " C")
318 elif temp < T_MIN:
319     al_t = "TEMP BAJA"
320     lista_alertas.append("Temperatura baja: " + str(temp) + " C")
321 else:
322     al_t = ""
323
324 if hum > H_MAX:
325     al_h = "HUMEDAD ALTA"
326     lista_alertas.append("Humedad alta: " + str(hum) + " %")
327 elif hum < H_MIN:
328     al_h = "HUMEDAD BAJA"
329     lista_alertas.append("Humedad baja: " + str(hum) + " %")
330 else:
331     al_h = ""
332
333 if mag >= A_BRUS:
334     al_m = "MOV BRUSCO"
335     lista_alertas.append("Movimiento brusco")
336 elif mag >= A_MOV:
337     al_m = "MOVIMIENTO"
338     lista_alertas.append("Movimiento detectado")
339 else:
340     al_m = ""
341
342 if panico_activo:
343     al_p = True
344     lista_alertas.append("PANICO ACTIVADO")
345 else:
346     al_p = False
347
348 if panico_activo:
```

```

349     _bz.freq(2800)
350     buzzer_on()
351 elif mag >= A_BRUS:
352     _bz.freq(2200)
353     buzzer_on()
354 elif mag >= A_MOV:
355     _bz.freq(1600)
356     buzzer_on()
357 elif temp > T_MAX or temp < T_MIN:
358     _bz.freq(1000)
359     buzzer_on()
360 elif hum > H_MAX or hum < H_MIN:
361     _bz.freq(700)
362     buzzer_on()
363 else:
364     buzzer_off()
365
366 if lista_alertas:
367     mensaje = "ALERTAS DEL SISTEMA\n\n"
368     for alerta in lista_alertas:
369         mensaje += "- " + alerta + "\n"
370     telegram(mensaje)
371
372 # =====
373 # SERIAL
374 # =====
375
376 def serial():
377     print("\n=====")
378     print(" IOT BIOMEDICO")
379     print("=====")
380     print(" Temperatura :", temp, "C")
381     print(" Humedad      :", hum, "%")
382     print(" Movimiento   :", round(mag, 3), "g")
383     print(" Estado Mov   :", emov)
384     print(" Panico       :", "SI" if al_p else "NO")
385     alarms = []
386     if al_t: alarms.append(al_t)
387     if al_h: alarms.append(al_h)
388     if al_m: alarms.append(al_m)
389     if al_p: alarms.append("PANICO")
390     if alarms:
391         print(" ALARMAS:", " | ".join(alarms))
392     else:
393         print(" Estado normal")
394     print("=====")
395
396 # =====
397 # PAGINA WEB
398 # =====
399

```

```

400 def html():
401     estado = "PANICO ACTIVO" if al_p else "SISTEMA NORMAL"
402     color_estado = "#ff0000" if al_p else "#22c55e"
403     pagina = ""
404     <!DOCTYPE html>
405     <html>
406     <head>
407         <meta charset='UTF-8'>
408         <meta http-equiv='refresh' content='3'>
409         <meta name='viewport' content='width=device-width, initial-
            scale=1.0'>
410         <title>IoT Biomedico</title>
411         <style>
412             body{ font-family:Arial; background:#0f172a; color:white;
413                 text-align:center; padding:20px; }
414             .card{ background:#1e293b; padding:20px; margin:15px auto;
415                 border-radius:15px; max-width:400px; }
416             .valor{ font-size:35px; font-weight:bold; }
417             .estado{ color:"" + color_estado + ""; font-size:28px;
418                 font-weight:bold; }
419         </style>
420     </head>
421     <body>
422         <h1>Monitor IoT Biomedico</h1>
423         <div class='card'>
424             <div>Temperatura</div>
425             <div class='valor'>"" + str(temp) + "" C</div>
426         </div>
427         <div class='card'>
428             <div>Humedad</div>
429             <div class='valor'>"" + str(hum) + "" %</div>
430         </div>
431         <div class='card'>
432             <div>Movimiento</div>
433             <div class='valor'>"" + str(round(mag, 2)) + "" g</div>
434             <div>"" + emov + ""</div>
435         </div>
436         <div class='card'>
437             <div>Estado</div>
438             <div class='estado'>"" + estado + ""</div>
439         </div>
440     </body>
441 </html>
442     ""
443     return pagina
444
445 # =====
446 # ATENDER CLIENTE WEB
447 # =====
448
449 def atender(conn):

```

```

450     try:
451         conn.settimeout(3)
452         request = conn.recv(1024)
453         if not request:
454             conn.close()
455             return
456         pagina = html()
457         pagina_bytes = pagina.encode("utf-8")
458         cabecera = (
459             "HTTP/1.1 200 OK\r\n"
460             "Content-Type: text/html; charset=utf-8\r\n"
461             "Content-Length: " + str(len(pagina_bytes)) + "\r\n"
462             "Connection: close\r\n"
463             "\r\n"
464         )
465         conn.send(cabecera.encode("utf-8"))
466         tam = 512
467         for i in range(0, len(pagina_bytes), tam):
468             conn.send(pagina_bytes[i:i+tam])
469             time.sleep_ms(5)
470     except Exception as e:
471         print("[WEB ERROR]", e)
472     finally:
473         try:
474             conn.close()
475         except:
476             pass
477
478     # =====
479     # CREAM SERVIDOR
480     # =====
481
482     def crear_servidor(ip):
483         try:
484             s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
485             s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
486             s.bind(("0.0.0.0", 80))
487             s.listen(2)
488             s.settimeout(0.2)
489             print("[WEB] http://" + ip)
490             return s
491         except Exception as e:
492             print("[WEB ERROR]", e)
493             return None
494
495     # =====
496     # WIFI
497     # =====
498
499     def conectar_wifi():
500         w = network.WLAN(network.STA_IF)

```

```
501     w.active(True)
502     print("[WiFi] Conectando...")
503     SSID      = "Laura"
504     PASSWORD  = "Laura12345"
505     w.connect(SSID, PASSWORD)
506     while not w.isconnected():
507         time.sleep(0.5)
508         print(".", end="")
509     print()
510     ip = w.ifconfig()[0]
511     print("[WiFi] Conectado")
512     print("[WiFi] IP:", ip)
513     print("[WEB] Abrir en navegador: http://" + ip)
514     return ip
515
516 # =====
517 # MAIN
518 # =====
519
520 def main():
521     global temp, hum, mag, emov
522
523     init_mpu()
524
525     print("[DHT] Estabilizando...")
526     time.sleep(2)
527
528     ip      = conectar_wifi()
529     servidor = crear_servidor(ip)
530
531     _bz.freq(2000)
532     buzzer_on()
533     time.sleep_ms(120)
534     buzzer_off()
535     time.sleep_ms(100)
536     _bz.freq(3000)
537     buzzer_on()
538     time.sleep_ms(120)
539     buzzer_off()
540
541     print("\n[OK] Sistema listo\n")
542
543     ciclo = 0
544
545     while True:
546         ciclo += 1
547         mag = leer_acc()
548         emov = clasificar(mag)
549
550         if ciclo % 6 == 0:
551             t, h = leer_dht()
```



```
552         if t is not None and h is not None:
553             temp = t
554             hum = h
555             print("[DHT OK]", temp, hum)
556         else:
557             print("[DHT] Manteniendo ultima lectura valida")
558
559     evaluar()
560     revisar_telegram()
561     serial()
562
563     if servidor:
564         try:
565             conn, addr = servidor.accept()
566             atender(conn)
567         except OSError:
568             pass
569         except Exception as e:
570             print("[WEB ERROR]", e)
571
572     time.sleep(0.5)
573
574 # =====
575 # INICIO
576 # =====
577
578 main()
```

12. Conclusión

El desarrollo de este sistema IoT basado en ESP32 permitió integrar diferentes tecnologías de monitoreo ambiental y detección de movimiento en una sola plataforma funcional. A través del uso de sensores como el DHT11/DHT22 y el MPU6050, fue posible supervisar variables importantes como temperatura, humedad y estado de movimiento en tiempo real, lo cual representa una solución útil para aplicaciones biomédicas y de monitoreo remoto.

Además, la implementación de un servidor web embebido y la integración con Telegram demostraron la capacidad del sistema para ofrecer acceso remoto, visualización de datos y generación de alertas multimodales de manera eficiente. El proyecto también permitió aplicar conceptos de electrónica digital, redes IoT y programación en MicroPython, fortaleciendo habilidades prácticas en sistemas embebidos y automatización.